# SQL Injection Attacks

---

## 1. Executive Summary & Lab Setup

This report documents a comprehensive security assessment and penetration testing exercise performed against the Damn Vulnerable Web Application (DVWA) environment. The primary focus of the assessment is identifying, analyzing, and exploiting SQL Injection (SQLi) vulnerabilities across three distinct security tiers: Low, Medium, and High.

## 2. Technical Documentation & Level Analysis

### Low Level

SQL Injection Payload Used:

```
Mahmoud' and 1=1 union select null,concat(user,0x0a,password) from
users #
```

Vulnerability Description & Analysis:

At the low security level, the application implements zero input sanitization, filtering, or validation. The user-supplied id parameter is concatenated directly into the SQL query string and processed via standard database functions (mysqli_query()). The inserted payload successfully closes the original string context, invokes a UNION operator to append a secondary query, and extracts administrative usernames along with their corresponding password hashes using concat(user,0x0a,password) from the users table.

## Medium Level

SQL Injection Payload Used:

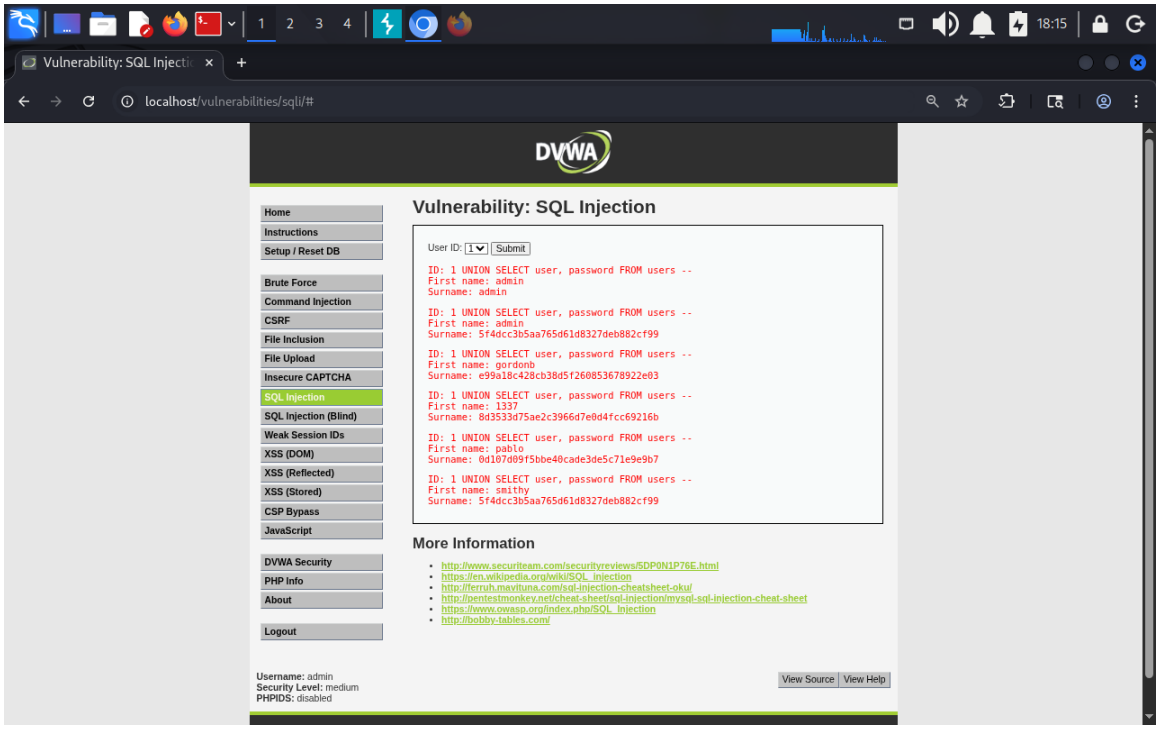
```
1 UNION SELECT user, password FROM users --
```

Vulnerability Description & Analysis:

The medium security level utilizes a dropdown selection box on the frontend interface instead of a free-text input field, coupled with basic defensive coding controls such as mysqli_real_escape_string() to constrain user interaction.

Bypass Methodology:

The frontend interface controls were bypassed by capturing and modifying the backend HTTP request parameters directly. Because the input parameter remained vulnerable to structural manipulation via integer-based injection without strict type-casting enforcement, the custom query successfully appended the UNION statement and retrieved the target database credentials.



## High Level

SQL Injection Payload Used:

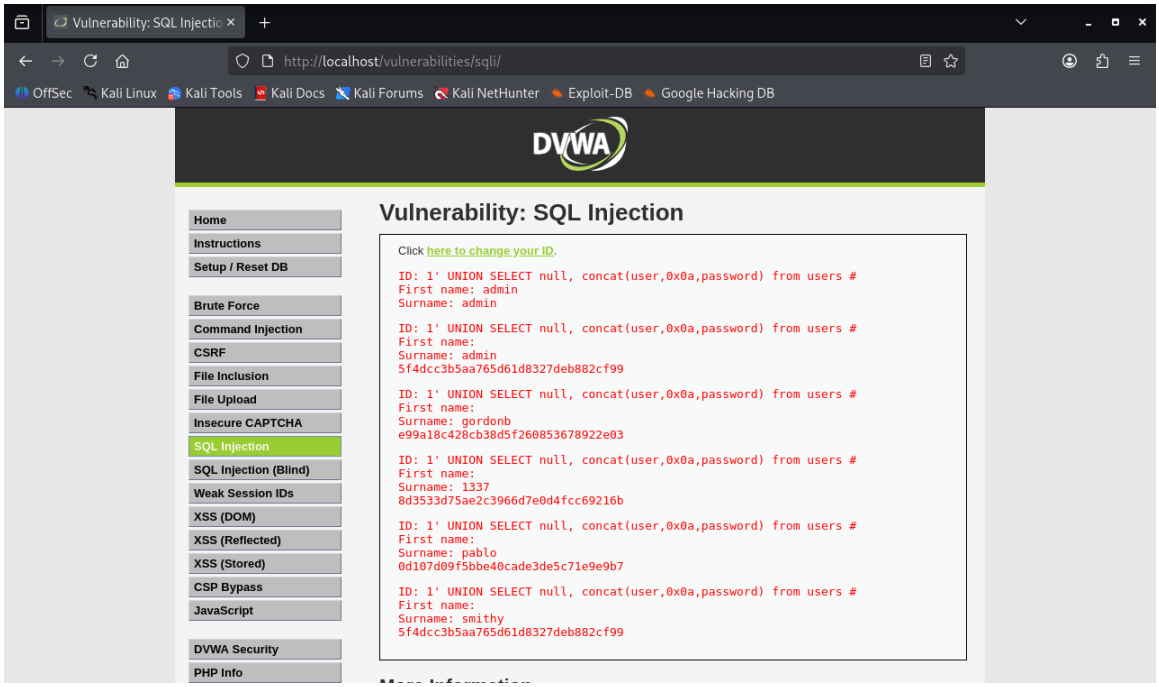
```
1' UNION SELECT null, concat(user,0x0a,password) from users #
```

Vulnerability Description & Analysis:

The high security level enforces rigorous input filtering, parameterized checks, and isolates user parameters from direct POST/GET query processing.

Bypass Methodology & Session Exploitation:

The vulnerability was successfully exploited by targeting the application's Session Management mechanism. Instead of accepting direct input fields, the high-security module passes and stores user-supplied identifiers via PHP sessions ($_SESSION variables) utilizing a secondary popup input window. By injecting the UNION payload into the session-managed ID parameter, standard direct input filters were circumvented, allowing the backend query to execute the concatenated SQL payload and dump the database records successfully.




# 4. Conclusion

This assessment successfully demonstrated how unvalidated inputs and improper query structuring can lead to full database compromise across multiple security tiers in the DVWA environment. Implementing robust input controls and parameterized queries is essential to defending modern web applications against unauthorized data extraction and maintaining information security integrity.